Chapter 18: Concurrency Control

Outline

Lock-Based Protocols

Deadlock Handling

Multiple Granularity

Insert and Delete Operations

Multiversion Schemes

Timestamp-Based Protocols

Validation-Based Protocols

Lock-Based Protocols

A lock is a mechanism to control concurrent access to a data item

Data items can be locked in two modes :

1. *exclusive (X) mode*. Data item can be both read as well as written. X-lock is requested using **lock-X** instruction.
2. *shared (S) mode*. Data item can only be read. S-lock is requested using **lock-S** instruction.

Lock requests are made to concurrency-control manager. Transaction can proceed only after request is granted.

Lock-Based Protocols (Cont.)

Lock-compatibility matrix

	S	X
S	true	false
X	false	false

A transaction may be granted a lock on an item if the requested lock is compatible with locks already held on the item by other transactions

Any number of transactions can hold shared locks on an item,

> but if any transaction holds an exclusive on the item no other transaction may hold any lock on the item.

If a lock cannot be granted, the requesting transaction is made to wait till all incompatible locks held by other transactions have been released. The lock is then granted.

The Two-Phase Locking Protocol

This is a protocol which ensures conflict-serializable schedules.

Phase 1: Growing Phase

transaction may obtain locks

transaction may not release locks

Phase 2: Shrinking Phase

transaction may release locks

transaction may not obtain locks

lock-S(A);

read *(A)*;

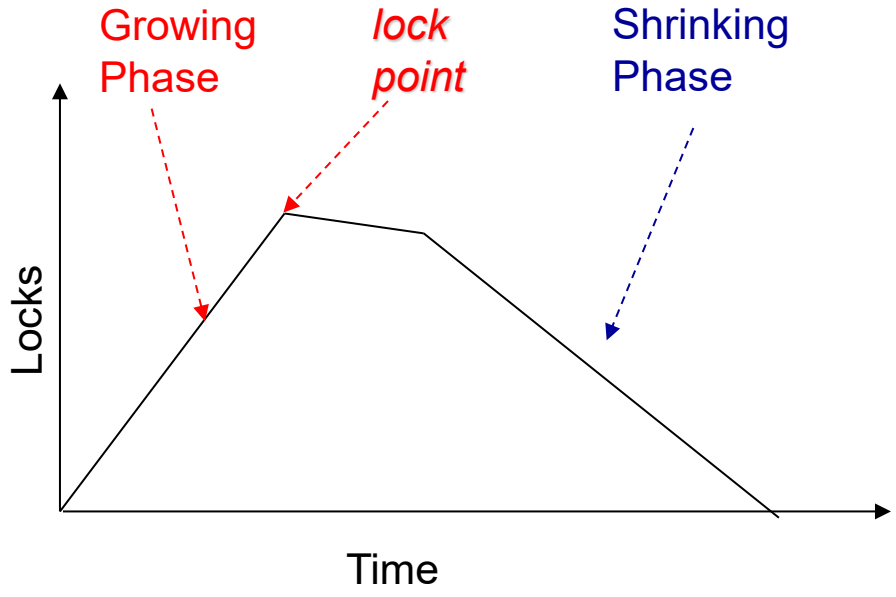
lock-S(B); ← ***lock point***

read *(B)*;

unlock*(A)*;

unlock(B);

display*(A+B)*



The Two-Phase Locking Protocol

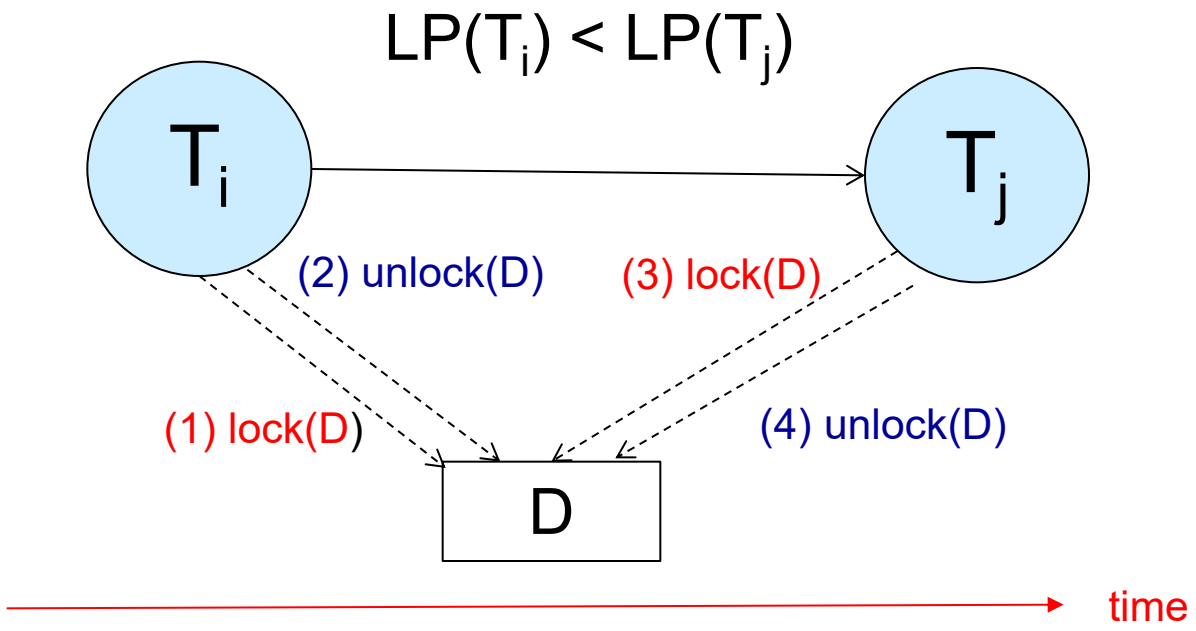
- Two-Phase Locking Protocol assures serializability.

  It can be proved that the transactions can be serialized in the order of their **lock points** (i.e. the point where a transaction acquired its final lock).

2PL - Proof by Contradiction

In the precedence graph corresponding to a schedule of a set of transactions T_1, T2, ..., T_n, if there is an arc from T_i to T_j , then $LP(T_i) < LP(T_j)$

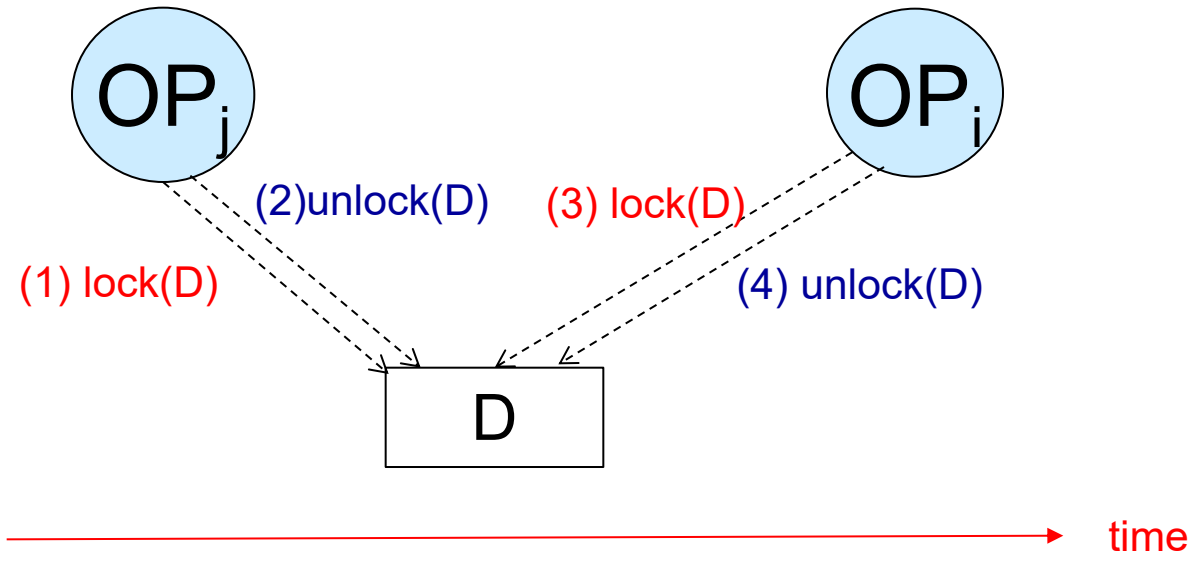
(LP : Lock Point)



2PL - Proof by Induction

Let T_i is the transaction with minimum lock point（LP）.

If there is an operation OP_j of another transaction T_j that blocks an operation OP_i of T_i …



$LP(T_j) < LP(T_i)$

The Two-Phase Locking Protocol (Cont.)

Extensions to **basic two-phase locking（基本两阶段封锁）** needed to ensure recoverability of freedom from cascading roll-back

Strict two-phase locking（严格两阶段封锁）: a transaction must hold all its exclusive locks till it commits/aborts.

- Ensures recoverability and avoids cascading roll-backs

Rigorous two-phase locking（强两阶段封锁）: a transaction must hold *all* locks till commit/abort.

- Transactions can be serialized in the order in which they commit.

Most databases implement rigorous two-phase locking, *but refer to it as simply two-phase locking*

The Two-Phase Locking Protocol (Cont.)

Two-phase locking is not a necessary condition for serializability.

There can be conflict serializable schedules that cannot be obtained if two-phase locking is used.

However, in the absence of extra information (e.g., ordering of access to data), two-phase locking is needed for conflict serializability .

Exercise

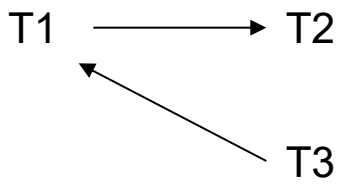
The following is a schedule of the transactions T1,T2,T3:

T1	T2	T3
write C		
	write C	
		write A
write A		

a) Draw the precedence graph of the schedule

b) Is the schedule conflict serializable?

c) Can the schedule be generated by the 2PL protocol?



Exercise(continue…)

The following is the schedule with lock and unlock operations.

T1	T2	T3
lock-X(C)		
write C		
unlock(C)		
	lock-X(C)	
	write C	
	unlock(C)	
		lock-X(A)
		write A
		unlock(A)
lock-X(A)		
write A		
unlock(A)		

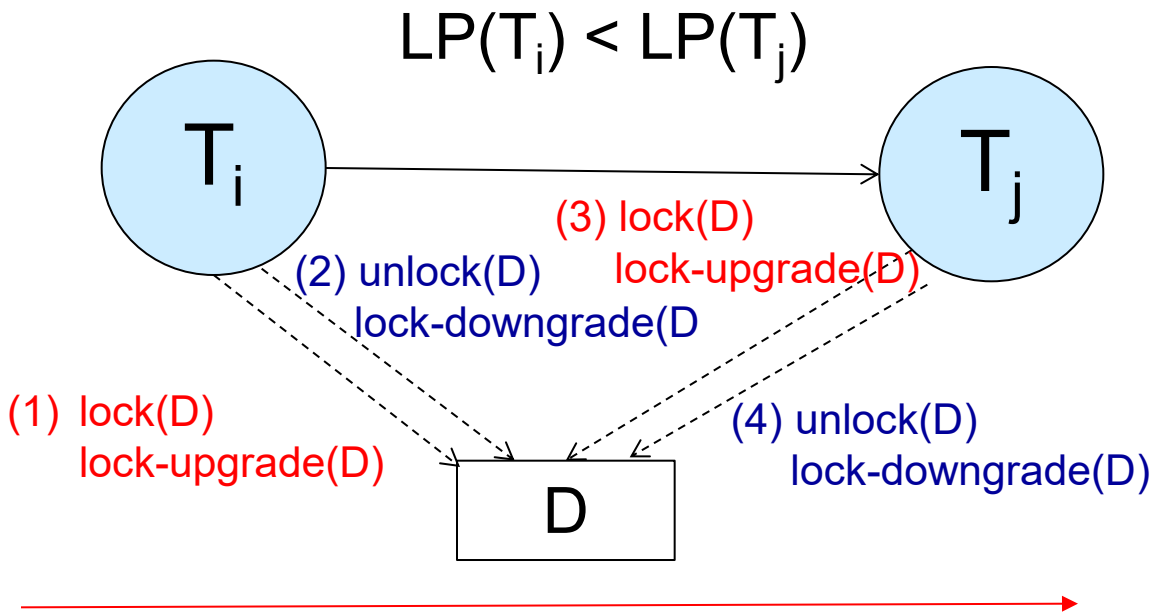
Lock Conversions

Two-phase locking with lock conversions:

- First Phase:
  - can acquire a lock-S or lock-X on a data item
  - can convert a lock-S to a lock-X (**lock-upgrade**)
- Second Phase:
  - can release a lock-S or lock-X
  - can convert a lock-X to a lock-S (**lock-downgrade**)

This protocol assures serializability.

(Please prove it !)



Automatic Acquisition of Locks

A transaction T_i issues the standard read/write instruction, without explicit locking calls.

The operation **read**(*D*) is processed as:

```
if Ti has a lock on D
  then
      read(D)
  else begin
      if necessary wait until no other
         transaction has a lock-X on D
      grant Ti a  lock-S on D;
      read(D)
    end
```

Automatic Acquisition of Locks (Cont.)

write(*D*) is processed as:

if T_i has a **lock-X** on *D*
then
write(*D*)
else begin
if necessary wait until no other trans. has any lock on *D*,
if T_i has a **lock-S** on *D*
then
upgrade lock on *D* to **lock-X**
else
grant T_i a **lock-X** on *D*
write(*D*)
end;

All locks are released after commit or abort

Implementation of Locking

A **lock manager** can be implemented as a separate process to which transactions send lock and unlock requests

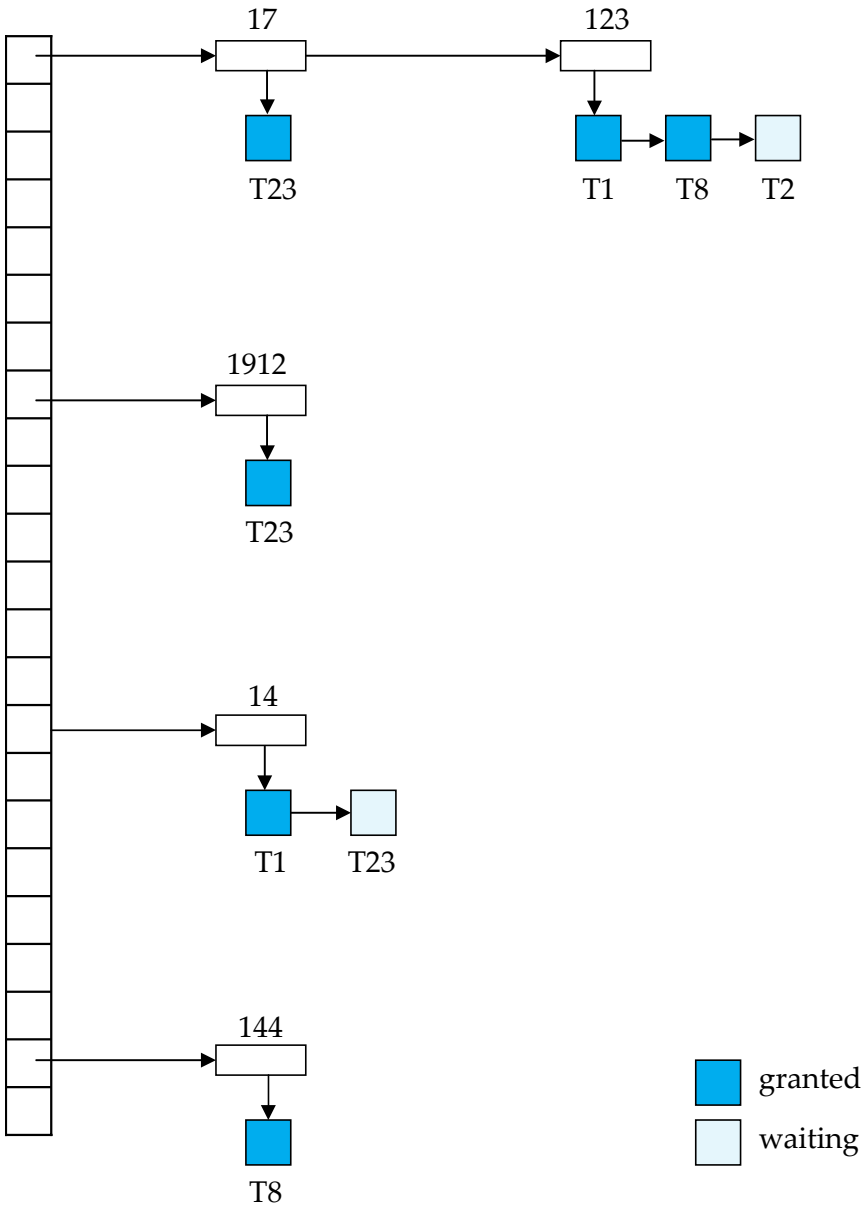
The lock manager replies to a lock request by sending a lock grant messages (or a message asking the transaction to roll back, in case of a deadlock)

The requesting transaction waits until its request is answered

The lock manager maintains a data-structure called a **lock table** to record granted locks and pending requests

The lock table is usually implemented as an in-memory hash table indexed on the name of the data item being locked

Lock Table



Lock table records granted locks and waiting requests

Lock table also records the type of lock granted or requested

New request is added to the end of the queue of requests for the data item, and granted if it is compatible with all earlier locks

Unlock requests result in the request being deleted, and later requests are checked to see if they can now be granted

If transaction aborts, all waiting or granted requests of the transaction are deleted

- lock manager may keep a list of locks held by each transaction, to implement this efficiently

Deadlock Handling

System is deadlocked if there is a set of transactions such that every transaction in the set is waiting for another transaction in the set.

Two-phase locking *does not* ensure freedom from deadlocks

Consider the following two transactions:

T_1:	write (X)	T_2:	write(Y)
	write(Y)		write(X)

Schedule with deadlock

T_1	T_2
lock-X on A	
write (A)	
	lock-X on B
	write (B)
	wait for **lock-X** on A
wait for **lock-X** on B	

Deadlock Handling

Deadlock prevention protocols ensure that the system will *never* enter into a deadlock state. Some prevention strategies :

Require that each transaction locks all its data items before it begins execution (predeclaration).

Impose partial ordering of all data items and require that a transaction can lock data items only in the order specified by the partial order (graph-based protocol).

Timeout-Based Schemes:

a transaction waits for a lock only for a specified amount of time. After that, the wait times out and the transaction is rolled back.

thus deadlocks are not possible

simple to implement; but starvation is possible. Also difficult to determine good value of the timeout interval.

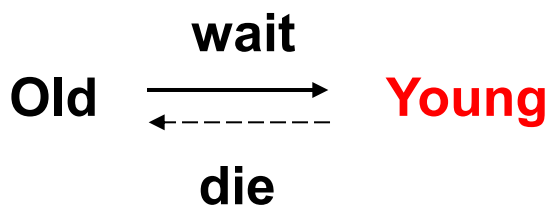
More Deadlock Prevention Strategies

Following schemes use transaction timestamps for the sake of deadlock prevention alone.

wait-die scheme — non-preemptive

older transaction may wait for younger one to release data item. Younger transactions never wait for older ones; they are rolled back instead.

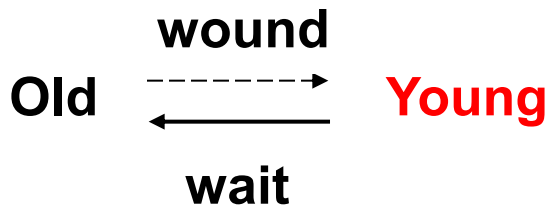
a transaction may die several times before acquiring needed data item



wound-wait scheme — preemptive

older transaction *wounds* (forces rollback) of younger transaction instead of waiting for it. Younger transactions may wait for older ones.

may be fewer rollbacks than *wait-die* scheme.



Both in wait-die and in wound-wait schemes, a rolled back transactions is restarted with its original timestamp. Older transactions thus have precedence over newer ones, and starvation is hence avoided.

Deadlock Detection

Deadlocks can be described as a ***wait-for*** graph, which consists of a pair $G = (V,E)$,

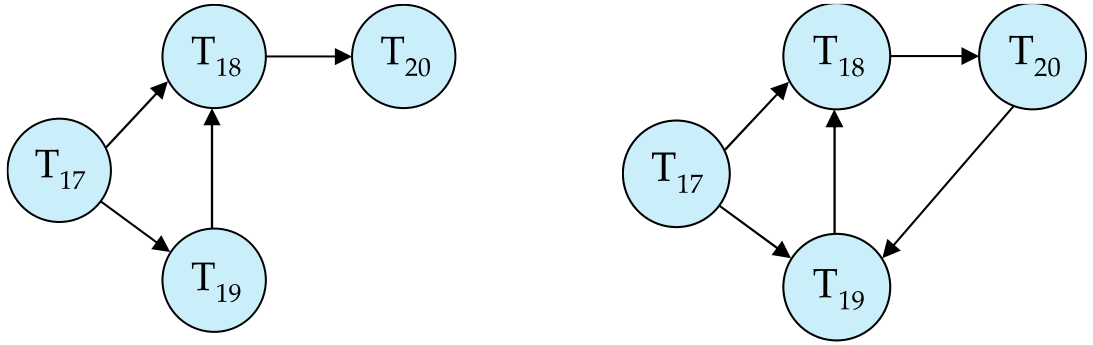
V is a set of vertices (all the transactions in the system)

E is a set of edges; each element is an ordered pair $T_i \rightarrow T_j$.

If $T_i \rightarrow T_j$ is in E, then there is a directed edge from T_i to T_j, implying that T_i is waiting for T_j to release a data item.

When T_i requests a data item currently being held by T_j, then the edge $T_i \ T_j$ is inserted in the wait-for graph. This edge is removed only when T_j is no longer holding a data item needed by T_i.

The system is in a deadlock state if and only if the wait-for graph has a cycle. Must invoke a deadlock-detection algorithm periodically to look for cycles.



Deadlock Recovery

When deadlock is detected :

Some transaction will have to rolled back (made a victim) to break deadlock. Select that transaction as victim that will incur minimum cost.

Rollback -- determine how far to roll back transaction

- Total rollback: Abort the transaction and then restart it.
- More effective to roll back transaction only as far as necessary to break deadlock.

Starvation happens if same transaction is always chosen as victim. Include the number of rollbacks in the cost factor to avoid starvation

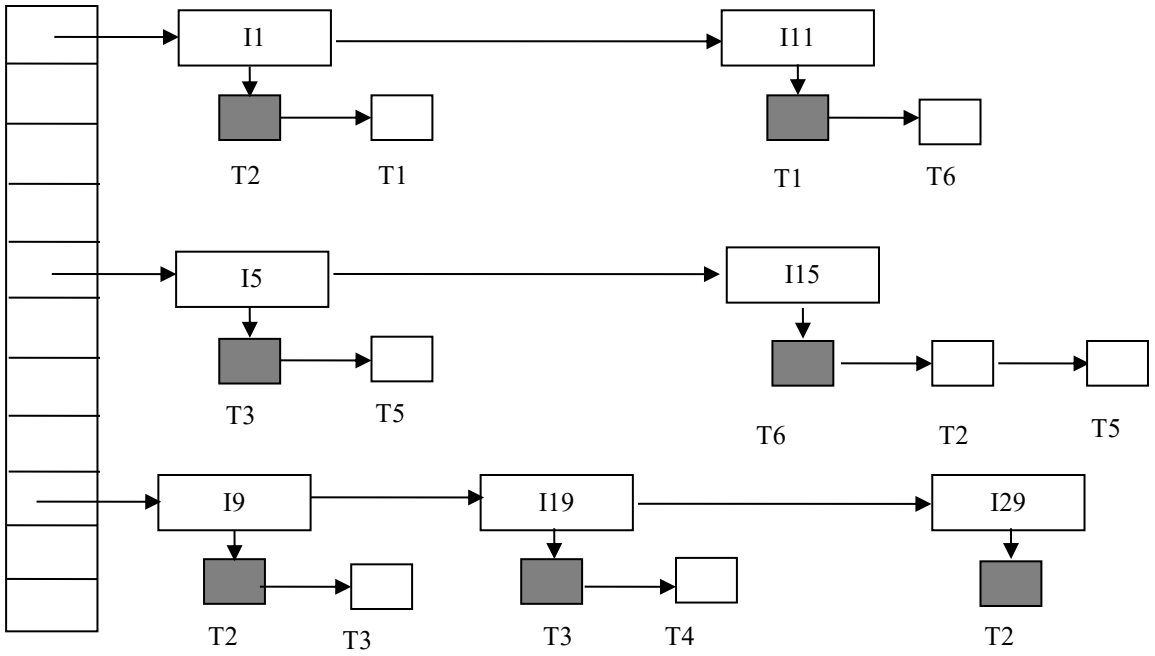
Exercise

Following figure shows an example of a lock table. There are 6 transactions(T1-T6) and 6 data items(I1, I11, I5, I15, I9, I19, I29). Granted locks are filled (black) rectangles, while waiting requests are empty rectangles.

a) **Which transaction are involved in deadlock?**

b) **To break the deadlock, which transaction should be rolled back?**

c) **What kind data structure can be helpful for a transaction to efficiently release all locks its holds when it commits?**



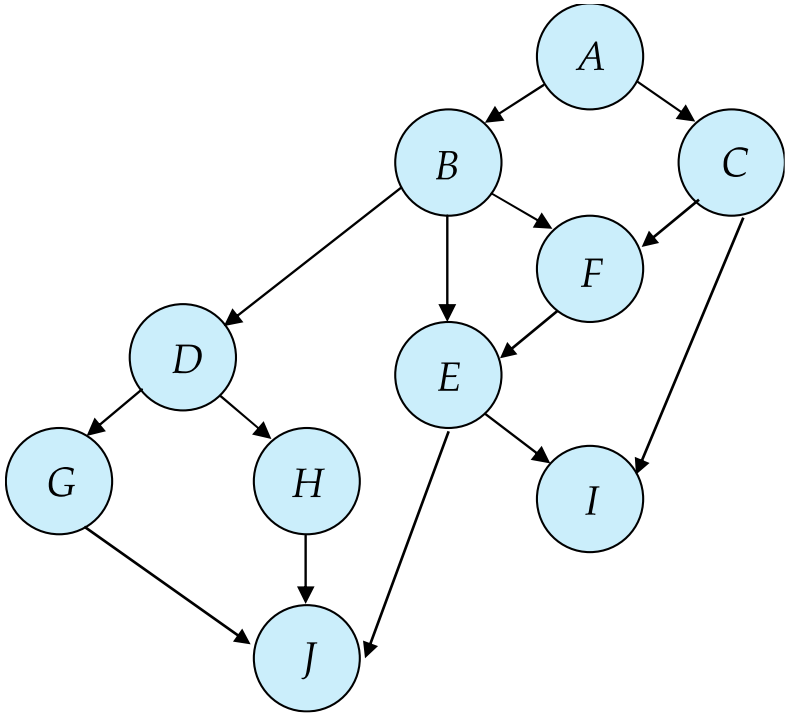
Graph-Based Protocols

Graph-based protocols are an alternative to two-phase locking

Impose a partial ordering $\rightarrow$ on the set **D** = $\{d_1, d_2, ..., d_h\}$ of all data items.

If $d_i \rightarrow d_j$ then any transaction accessing both d_i and d_j must access d_i before accessing d_j.

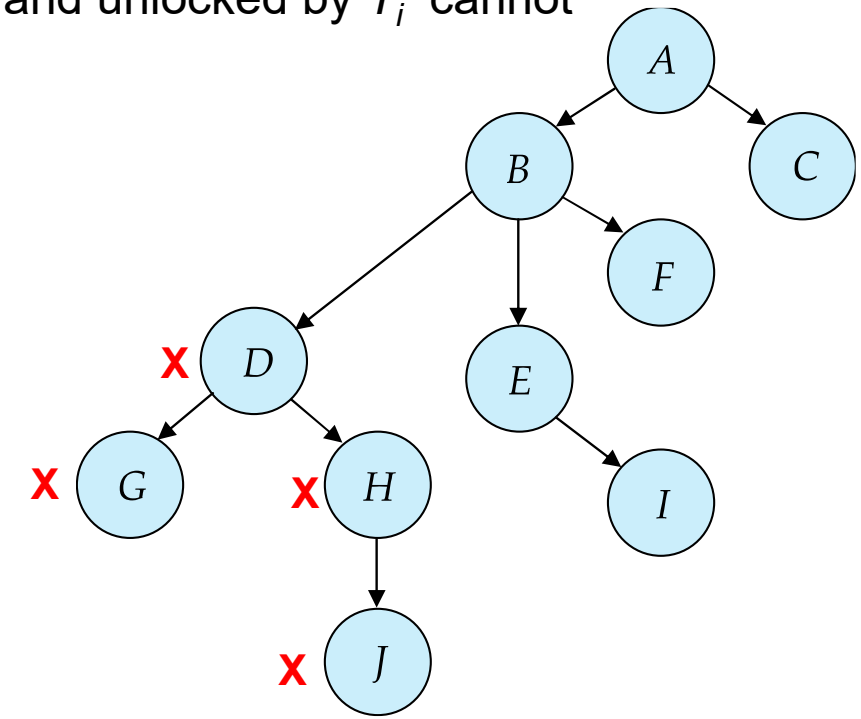
Implies that the set **D** may now be viewed as a directed acyclic graph, called a *database graph*.



Tree Protocol

The tree-protocol is a simple kind of graph protocol.

1. Only exclusive locks are allowed.
2. The first lock by T_i may be on any data item. Subsequently, a data Q can be locked by T_i only if the parent of Q is currently locked by T_i.
3. Data items may be unlocked at any time.
4. A data item that has been locked and unlocked by T_i cannot subsequently be relocked by T_i



Graph-Based Protocols (Cont.)

The tree protocol ensures conflict serializability as well as freedom from deadlock.

Advantages

Unlocking may occur earlier in the tree-locking protocol than in the two-phase locking protocol.

- shorter waiting times, and increase in concurrency

protocol is deadlock-free

- no rollbacks are required

Disadvantages

Protocol does not guarantee recoverability or cascade freedom

- Need to introduce commit dependencies to ensure recoverability

Transactions may have to lock more data items than needed.

- increased locking overhead, and additional waiting time
- potential decrease in concurrency

Schedules not possible under two-phase locking are possible under tree protocol, and vice versa.

Multiple Granularity(多粒度)

Allow data items to be of various sizes and define a hierarchy of data granularities, where the small granularities are nested within larger ones

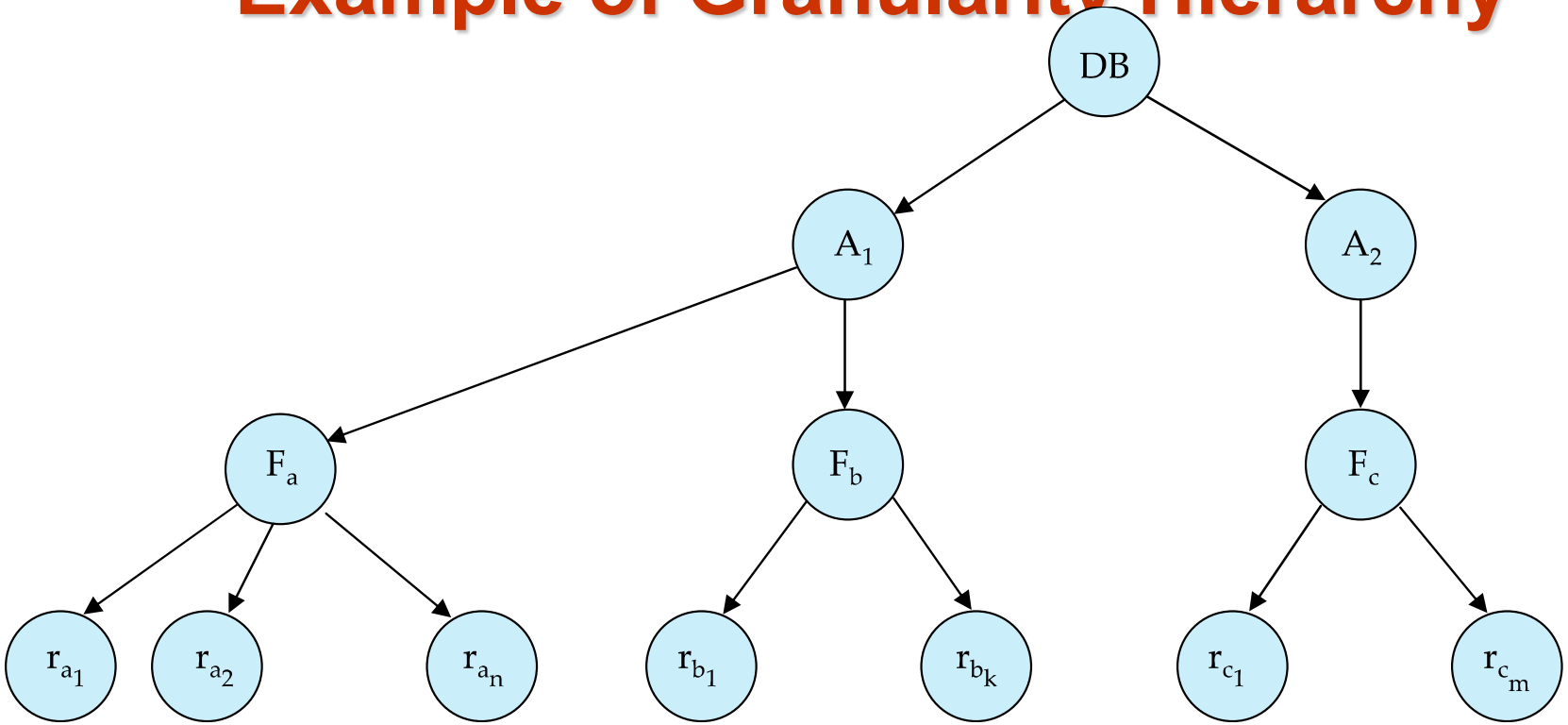
Can be represented graphically as a tree (but don't confuse with tree-locking protocol)

When a transaction locks a node in the tree *explicitly*, it *implicitly* locks all the node's descendents in the same mode.

Granularity of locking (level in tree where locking is done):

- **fine granularity（细粒度）**(lower in tree): high concurrency, high locking overhead
- **coarse granularity（粗粒度）** (higher in tree): low locking overhead, low concurrency

Example of Granularity Hierarchy



The levels, starting from the coarsest (top) level are

- *database*
- *area*
- *File (table)*
- *record*

Intention Lock Modes

In addition to S and X lock modes, there are three additional lock modes with multiple granularity:

intention-shared (IS): indicates explicit locking at a lower level of the tree but only with shared locks.

intention-exclusive (IX): indicates explicit locking at a lower level with exclusive or shared locks

shared and intention-exclusive (SIX): the subtree rooted by that node is locked explicitly in shared mode and explicit locking is being done at a lower level with exclusive-mode locks.

intention locks allow a higher level node to be locked in S or X mode without having to check ***all*** descendent nodes.

Compatibility Matrix with Intention Lock Modes

The compatibility matrix for all lock modes is:

	IS	IX	S	SIX	X
IS	true	true	true	true	false
IX	true	true	false	false	false
S	true	false	true	false	false
SIX	true	false	false	false	false
X	false	false	false	false	false

Multiple Granularity Locking Scheme

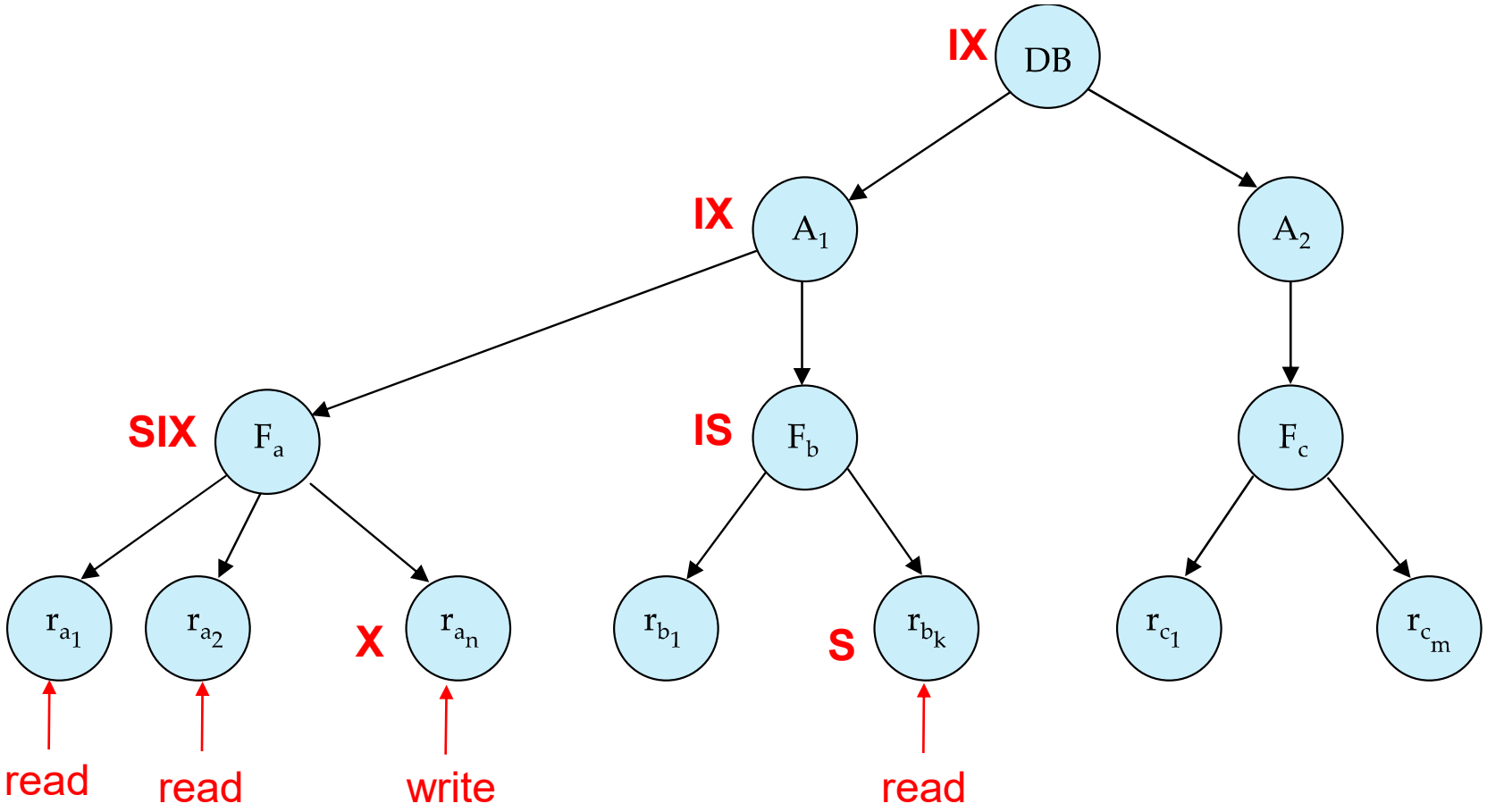
Transaction T_i can lock a node Q, using the following rules:

1. The lock compatibility matrix must be observed.
2. The root of the tree must be locked first, and may be locked in any mode.
3. A node Q can be locked by **T_i in S or IS** mode only if the parent of Q is currently locked by T_i in either IX or IS mode.
4. A node Q can be locked by **T_i in X, SIX, or IX** mode only if the parent of Q is currently locked by T_i in either IX or SIX mode.
5. T_i can lock a node only if it has not previously unlocked any node (that is, T_i is two-phase).
6. T_i can unlock a node Q only if none of the children of Q are currently locked by T_i.

Observe that locks are acquired in root-to-leaf order, whereas they are released in leaf-to-root order.

Lock granularity escalation: in case there are too many locks at a particular level, switch to higher granularity S or X lock

An Example of Multiple Granularity Locking



Insert and Delete Operations

If two-phase locking is used :

A **delete** operation may be performed only if the transaction deleting the tuple has an exclusive lock on the tuple to be deleted.

A transaction that **inserts** a new tuple into the database is given an X-mode lock on the tuple

Insertions and deletions can lead to the **phantom phenomenon**.

A transaction that scans a relation

- (e.g., find sum of balances of all accounts in Perryridge)

and a transaction that inserts a tuple in the relation

- (e.g., insert a new account at Perryridge)

(conceptually) conflict in spite of not accessing any tuple in common.

If only tuple locks are used, non-serializable schedules can result

- E.g. the scan transaction does not see the new account, but reads some other tuple written by the update transaction

Insert and Delete Operations (Cont.)

The transaction scanning the relation is reading information that indicates what tuples the relation contains, while a transaction inserting a tuple updates the same information.

The conflict should be detected, e.g. by locking the information.

One solution:

Associate a data item with the relation, to represent the information about what tuples the relation contains.

Transactions scanning the relation acquire a shared lock in the data item,

Transactions inserting or deleting a tuple acquire an exclusive lock on the data item. (Note: locks on the data item do not conflict with locks on individual tuples.)

Above protocol provides very low concurrency for insertions/deletions.

Index locking protocols provide higher concurrency while preventing the phantom phenomenon, by requiring locks on certain index buckets.

Index Locking Protocol

Index locking protocol:

Every relation must have at least one index.

A transaction can access tuples only after finding them through one or more indices on the relation

A transaction T_i that performs a **lookup** must lock all the index leaf nodes that it accesses, in S-mode

- Even if the leaf node does not contain any tuple satisfying the index lookup (e.g. for a range query, no tuple in a leaf is in the range)

A transaction T_i that **inserts, updates or deletes** a tuple t_i in a relation r

- must update all indices to r
- must obtain exclusive locks on all index leaf nodes affected by the insert/update/delete

The rules of the two-phase locking protocol must be observed

Guarantees that phantom phenomenon won't occur

Multiversion Concurrency Control Schemes

Multiversion schemes keep old versions of data item to increase concurrency.

Each successful **write** results in the creation of a new version of the data item written.

Use timestamps to label versions.

When a **read**(*Q*) operation is issued, select an appropriate version of *Q* based on the timestamp of the transaction, and return the value of the selected version.

Read only transactions never have to wait as an appropriate version is returned immediately for every read operation..

Q: <Q1, 1> , <Q2, 2> , <Q3, 3>

Multiversion Two-Phase Locking

Differentiates between read-only transactions and update transactions

SET TRANSACTIOIN READ ONLY;

SET TRANSACTION READ WRITE;

Update transactions

Acquire read and write locks

Hold all locks up to the end of the transaction. That is , follow rigorous two-phase locking.

Each successful **write** results in the creation of a new version of the data item written.

Each version of a data item has a single timestamp whose value is obtained from a counter **ts-counter** that is incremented during commit processing.

Read-only transactions

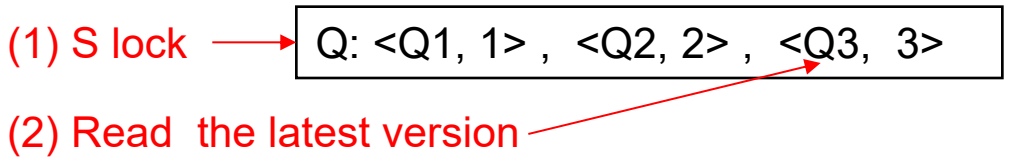
Assigned a timestamp by reading the current value of **ts-counter** before they start execution.

When a read-only transaction T_i issues a read(Q), the value returned is the contents of the version whose timestamp is the largest timestamp less than or equal to $TS(T_i)$

Multiversion Two-Phase Locking (Cont.)

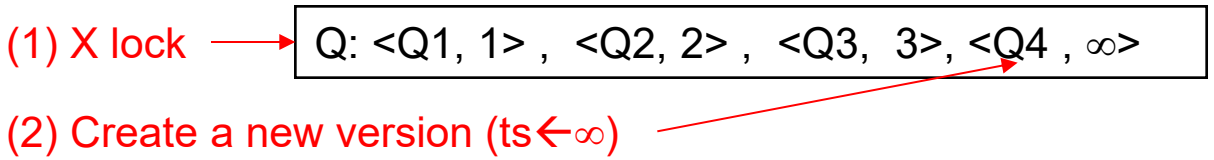
When an update transaction wants to read a data item:

it obtains a S lock on it, and reads the latest version.



When an update transaction wants to write an item

it obtains X lock on; it then creates a new version of the item and sets this version's timestamp to ∞.

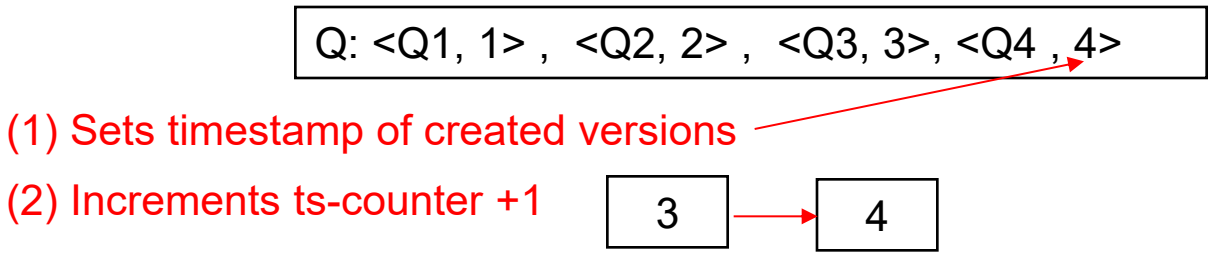


When update transaction T_i completes, commit processing occurs:

T_i sets timestamp on the versions it has created to **ts-counter** + 1

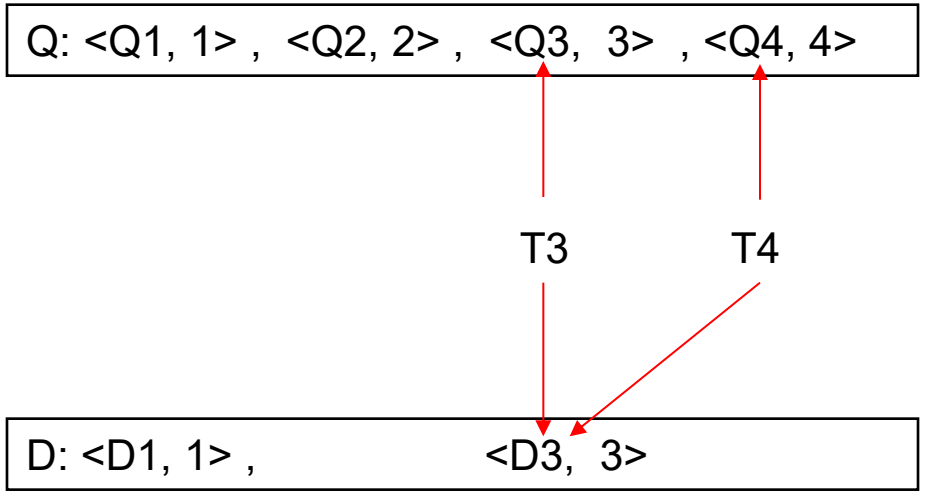
T_i increments **ts-counter** by 1

Unlock all locks it holds



Multiversion Two-Phase Locking (Cont.)

When a read-only transaction T_i issues a read(Q), the value returned is the contents of the version whose timestamp is the largest timestamp less than or equal to $TS(T_i)$



Only serializable schedules are produced.

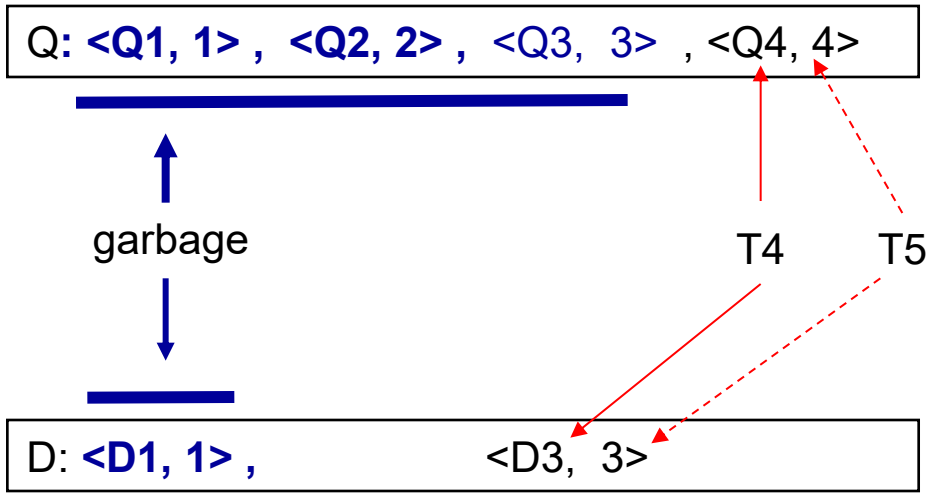
MVCC: Implementation Issues

Creation of multiple versions increases storage overhead

- Extra tuples
- Extra space in each tuple for storing version information

Obsolete versions should be garbage collected

- Keep the youngest version Q_k among all versions whose timestamp is less than or equal to the timestamp of the oldest read-only transaction in the system, all other versions older than the Q_k can be deleted.



End of Chapter 18